Improving ICU Patient Bed Moves Policies with Reinforcement Learning Based Simulation-Optimisation

Geraint I. Palmer-Liyu Mark Tuson Virginia Ahedo Paul R. Harper
Mathew Pugh James Williams Matt Morgan

May 22, 2026

1 Introduction

2 Background

TODO

Here we need some background on the ICU department at the Heath; background and literature on on ICU bed moves, why we move patients, why it's bad to move patients, etc.

3 ICU Description

The ICU at UHW consists of 35 beds across 4 wards (named A3 North, A3 South, B3 North, and B3 South), two single isolation units, and one double isolation unit. One bed is generally ring-fenced, left unoccupied, for emergencies. In each ward there is a partial wall partitioning the beds into two halves. This is shown in Figure 1. There are two additional units, Complex Care and the Post Anaesthetic Care Unit (PACU), which generally accommodate their own patient types (complex care and post anaesthetic patients respectively), spare beds can be used as surge, or overflow, for general ICU patients when needed.

The model will consider three types of patient, categorised as Stage 2, Stage 3, or Stage 3-I patients:

- Stage 2: these are the least severe patients. When two Stage 2 patients occupy neighbouring beds, both can be cared for by the same FTE nurse.
- Stage 3: these patients require one to one attention from nurses, regardless of patients in neighbouring beds;
- Stage 3-I: these patients require isolation and one to one attention. When an isolation unit is available, they must be admitted there. If an isolation unit is occupied by a Stage 2 or Stage 3 patient, and non-isolation bed is available, then the other patient must be moved to the available bed and the new Stage 3-I patient must be admitted to the isolation unit. If they cannot be placed in an isolation unit at all, they are placed in a shared unit, and will require their own FTE nurse similar to Stage 3 patients. They will also incur a penalty cost for each time unit they are placed in a shared block.

In order to make the problem tractable, we make the following simplifications and assumptions in the model:

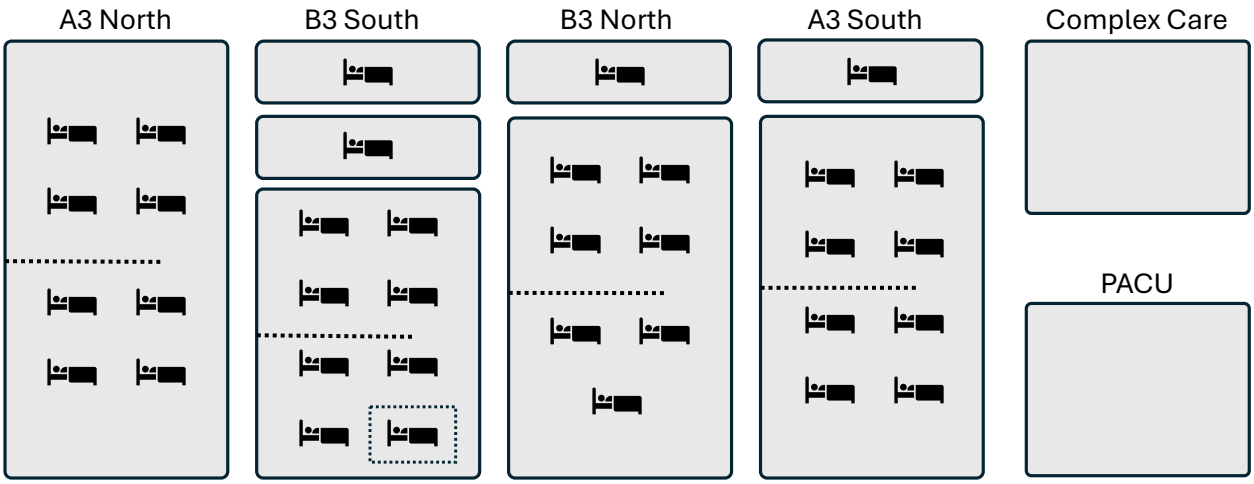


Figure 1: Representation of the four ICU wards. Solid blocks represent wards and isolation units, dotted lines represent partial walls within a ward, and the dotted box represents the ring-fenced bed.

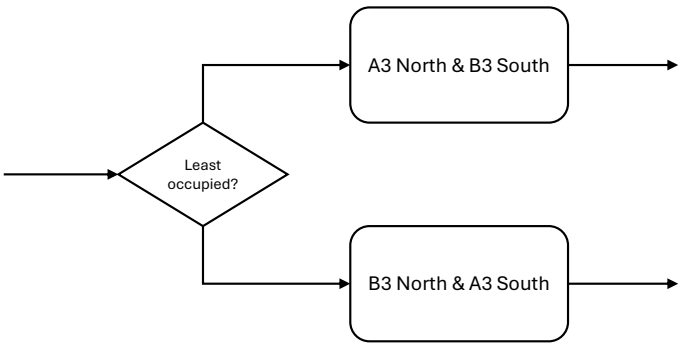
- the double isolation unit is treated as two separate isolation units. The only time this assumption may be violated would be if occupied by a patient with a contagious disease, in which case the second bed can then only be occupied by a patient with the same contagious disease; This is an uncommon enough occurrence to ignore for modelling purposes.
- general ICU patients are not admitted to Complex Care or PACU, if there are beds available in the other four wards. However, Stage 2 patients may be moved there as surge. This is generally undesirable, and so incurs some penalty;
- a nurse cannot care for two adjacent Stage 2 patients if there is a partial wall between the beds;
- the ring-fenced bed is never used, it will not be modelled.

Data analysis shows that:

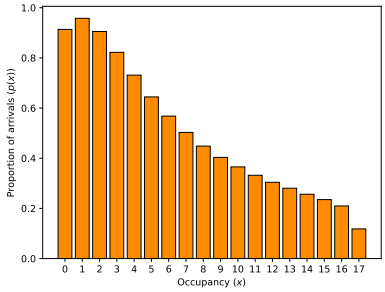
TODO

Parametrise these properly, referring back to Section 2. We can do some data analysis here.

- Stage 2 patients arrive according to the Poisson process with rate $\lambda_0 = 3.0$,
- Stage 3 patients arrive according to the Poisson process with rate $\lambda_1 = 2.0$,
- Stage 3-I patients arrive according to the Poisson process with rate $\lambda_2 = 1.0$,
- Stage 2 patients' lengths of stay are Exponentially distributed with rate $\mu_0 = 0.3$,
- Stage 3 patients' lengths of stay are Exponentially distributed with rate $\mu_1 = 0.7$,
- Stage 3-I patients' lengths of stay are Exponentially distributed with rate $\mu_2 = 0.4$,
- Stage 2 patients deteriorate into Stage 3 patients with rate δ_{01},
- Stage 3 patients deteriorate into Stage 3-I patients with rate δ_{12},
- Stage 3 patients improve into Stage 2 patients with rate δ_{10},
- Stage 3-I patients improve into Stage 3 patients with rate δ_{21}.



(a) Representation of the load-balancing arrival discipline for the two identical sub-problems.



(b) The proportion of the arrivals entering a single component, given the component's occupancy.

Figure 2: Problem decomposition analysis

4 Problem Decomposition

The 35 bed ICU problem can be decomposed into two sub-problems by introducing another assumption. Ignoring the ring-fenced bed, we can grouping wards A3 North and B3 South, and wards B3 North and A3 South, into two identical components each containing 17 beds. We then introduce the assumption that newly arriving patients are sent to the component with the most free beds. This corresponds to a *load-balancing* arrival discipline, a join-shortest-queue discipline that takes into consideration the number of occupied servers as well as queue length [6, 5], which, due to Poisson thinning, can be modelled using single queue analysis (SQA) with a state-dependent arrival rate. This is shown in Figure 2a.

The state-dependent arrival rate can be found from a preliminary simulation, built using Ciw [8], and parametrised as above, and observing the proportion of arrivals that enter each component at each state. The resulting proportions are given in Figure 2b. SQA now gives us that the arrival rate for each component, when the component's occupancy is x, is $\lambda_2 p(x)$, $\lambda_3 p(x)$, $\lambda_{3I} p(x)$.

We can now analyse each decomposed sub-problem separately, however each of our sub-problems are identical, so only one needs to be considered. The rest of the paper will now consider just one of the decomposed sub-problems, consisting of an ICU with two wards of 7 and 8 beds, and two isolation units, giving 17 beds in total. The 8 bed ward is partitioned in two by the partial wall, while the 7 bed wall is partitioned intro groups of 3 and 4 beds by its partial wall.

5 MDP Formulation

We will consider the bed move policy problem as a Markov Decision Process (MDP), which is a tuple (S, A, P, R), with states $s \in S$, actions $a \in A$, transition probabilities $p_{s,s',a} \in P$, and rewards $r_{s,s',a} \in R$. This is a discrete time MDP, with actions being taken at the discrete events when a new patient is admitted. The approach taken to find the optimal policy will be reinforcement learning, a model-free approach, and so the probabilities P need not be defined, and transitions are taken care of with a simulation model. The following sections describe each component.

5.1 States

The decomposed ICU ward contains 17 beds, which can be partitioned into 4 blocks, shown in Figure 3. These blocks are defined by physical walls: beds B00 to B07 are in one room, beds B08 to B14 are in another room, and beds B15 and B16 are in isolation units. Partial walls do not count as ward separations. These

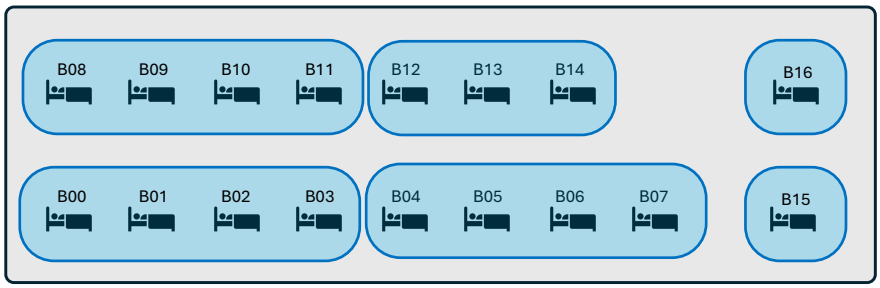


Figure 3: Representation of the 17 ICU beds partitioned into 4 blocks.

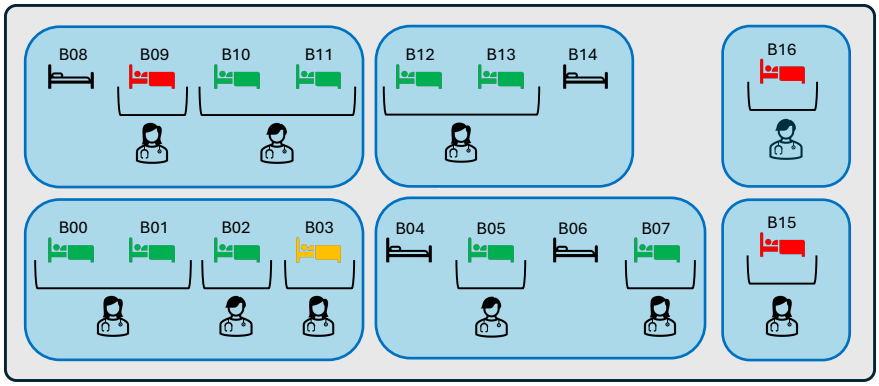


Figure 4: Representation of how nurses can share adjacent Stage 2 patients.

blocks will determine the cost of a bed-move: intra-block moves are easier than inter-block moves, and their cost or penalty will reflect this.

Within a block the location of a patient matters: if two Stage 2 patients are in adjacent beds, it is possible for one FTE nurse to care for both. This is shown in Figure 4. Therefore the states need to hold information on both the patient type, and which particular bed within a block the patient is in. However, both isolation units are indistinguishable from one another, so there is no need to differentiate these.

Define the ward state by a 3×16 matrix M, where rows correspond to patient types (where $j = 0$ corresponds to Stage 2 patients, $j = 1$ to Stage 3 patients, and $j = 2$ to Stage 3-I patients), and columns corresponding to bed numbers (except for column 15, which corresponds to both isolation units). Then its integer entries M_{ij} denote the number of category j patients currently occupying a bed i. Note that for columns 0 to 14 the columns sums should be no more than 1, as they correspond to single beds, while for column 15 the sum should be no more than 2, as it corresponds to both isolation units. Let $\mathcal{M}$ denote the set of all possible matrices M.

As an example, consider the ICU state given in Figure 4, where green, amber and red beds indicate beds occupied by Stage 2, Stage 3 and Stage 3-I patients respectively, and black beds are unoccupied. This corresponds to the ward state:

$$M^\star = \left(\begin{array}{cccc:cccc:cccc:cccc} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 2 \end{array}\right) \tag{1}$$

Ward state changes are caused by newly arriving patients of different categories being placed into beds in specific blocks, patient discharges, patient deterioration or improvement (a patient changing type), and

moving a patient from one bed to another.

Actions only take place when a new patient arrives to the ward, and this patient can be either Stage 2, Stage 3, or Stage 3-I (category 0, 1, or 2). Therefore the states of the MDP also need to contain information about the newly arriving patient. Note that Stage 2 patients cannot arrive when the ward is full, while Stage 3 and Stage 3-I patients can arrive when the ward is full, only if there is a Stage 2 patient present for them to displace. Let

$$s = (M, p)$$

represent the state of the MDP, where M is a 3×16 matrix representing the ward state, and $p \in \{0, 1, 2\}$ indicate the category of the arriving patient. Note that we only need states where a patient can arrive.

Proposition 1. *Let S be the set of all possible states $s = (M, p)$. Then $|S| = 32,111,460,252$.*

Proof. First consider $|\mathcal{M}|$, the number of matrices M that exist.

- For each of the first 15 columns representing beds not in an isolation unit, $M_{i,j} \in \{0, 1\}$, and the column sum $M_{\cdot,j} \leq 1$. There are four ways to satisfy this:

$$\left\{ \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\}$$

  Each column is independent of the rest, and so there are 4^{15} configurations for these columns.

- For the final column, representing the isolation units, $M_{i,15} \in \{0, 1, 2\}$, and the column sum $M_{\cdot,15} \leq 2$. There are ten ways to satisfy this:

$$\left\{ \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix} \right\}$$

  This column is independent of the others, and so there are 10×4^{15} possible matrices M representing valid system states.

However, not all states admit an arriving patient:

- Stage 2 patients cannot arrive if the ward is full. There are 3 ways in which each of the first 15 columns can be full, and 6 ways the final column can be full, and so there are 6×3^{15} that can be discarded for when $p = 0$.
- Stage 3 and 3-I patients can arrive if the system is full, as long as there is a Stage 2 patient present to displace and send to surge. There are 2 ways in which each of the first 15 columns can be full without a Stage 2 patient, and 3 ways for the final column, therefore there are 3×2^{15} states that can be discarded when $p \in \{1, 2\}$.

Therefore the total number of states $s = (M, p)$ is:

$$\begin{aligned} |S| &= (3 \times 10 \times 4^{15}) - (2 \times 3 \times 2^{15}) - (6 \times 3^{15}) \\ &= 32,111,460,252 \end{aligned}$$

□

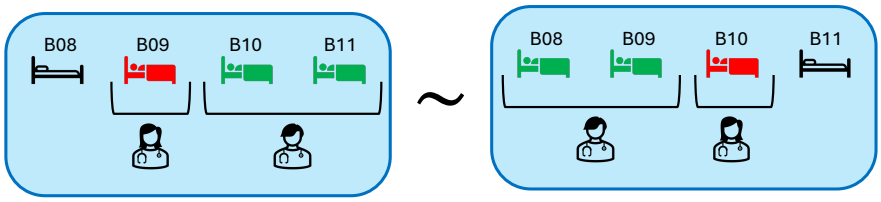


Figure 5: Example of an equivalence in a block of 4 beds.

5.1.1 State Equivalences

The state matrices exhibit some equivalences. In the three blocks of 4, and the block of 3, reversed configurations are equivalent up to a bed labelling. This is shown in Figure 5. Similarly, as the two blocks of 4 beds that are located in the same ward are identical, then swapping the configurations result in equivalent configurations up to a labelling.

Therefore define four column-transformation operations on $\mathcal{M}$ corresponding to these equivalences:

- T_1: reversing columns 0 to 3;
- T_2: reversing columns 4 to 7;
- T_3: reversing columns 8 to 11;
- T_4: reversing columns 12 to 14;
- T_5: swapping the block of columns 0–3 with the block 4–7.

We will consider all combinations of compositions of these transforms, and it will be useful later to understand their inverses. Note that all five of these transforms are involutions, that is they are their own inverse, and applying the same transform twice will give the original matrix. As the first four concern non-overlapping columns, then they are commutative under composition, and so any combination of these, P is also an involution [3, p. 22], and $P^{-1} = P$. This is not true however composing T_5 and T_1 or T_2 as they affect the same columns. However, if we T_5 was applied first or last, that is $P = \tilde{T} \circ T_5$ or $Q = T_5 \circ \tilde{T}$ where $\tilde{T}$ is some combination of the other transforms, then we can use that permutation as the inverse if we apply and re-apply T_5:

$$T_5 \circ P \circ T_5 = T_5 \circ (\tilde{T} \circ T_5) \circ T_5 = T_5 \circ \tilde{T} \circ I = T_5 \circ \tilde{T} = T_5^{-1} \circ \tilde{T}^{-1} = P^{-1}, \text{ or} \tag{2}$$

$$T_5 \circ Q \circ T_5 = T_5 \circ (T_5 \circ \tilde{T}) \circ T_5 = I \circ \tilde{T} \circ T_5 = \tilde{T} \circ T_5 = \tilde{T}^{-1} \circ T_5^{-1} = Q^{-1} \tag{3}$$

which are the exact inverse of P and Q [3, p. 18].

Now define a relation $\sim$ on $\mathcal{M}$ such that $M_1 \sim M_2$ if M_2 is obtained by applying some sequence of the column-transformations above on M_1. It is clear that $\sim$ satisfies reflexivity, symmetry, and transitivity, and is therefore an equivalence relation, and so $\mathcal{M}$ can be partitioned into disjoint equivalence classes $[M_1] = \{M_2 \in \mathcal{M} \mid M_2 \sim M_1\}$. Now define $\mathcal{M}^\star \subseteq \mathcal{M}$ to be the set containing exactly one representative matrix from each equivalence class. How this representative matrix is chosen is detailed in Section 6.1.1. Hence, we can also define $S^\star$ as the set of all possible states $s = (M, p)$ where $M \in \mathcal{M}^\star$, that is the set of all representative states.

Proposition 2. *A list of all elements of an equivalence class $[M]$ can be generated by applying all combinations of the transformations in $\{T_1, T_2, T_3, T_4, T_5\}$ to M, where the transforms in each composition are ordered by strictly increasing indices.*

This means, for example, that $T_1 \circ T_3 \circ T_4$ is evaluated, while an alternative ordering like $T_3 \circ T_1 \circ T_2$ is excluded. $T_1 \circ T_4 \circ T_1 \circ T_2$ is also excluded as we do not repeat transformations.

Proof. By definition $[M]$ contains all elements that can be reached from M by applying any combination of transforms in $\{T_1, T_2, T_3, T_4, T_5\}$, including repeats, in any order.

- First consider a permutation P that does not use T_5, the block-swap transformation. As all other transforms commute, we can apply the transforms in any order without changing P, and so we can obtain P by applying the transforms by strictly increasing indices. As for repeated application of any of the transforms, as they are involutions, any two consecutive repeated applications of the same transform cancel out, so P needs at most one application of each transform. Therefore P will be found by the generation scheme.
- Now consider a permutation Q that does contain at least one application of T_5.

  The only transforms that do not commute with T_5 are T_1 and T_2 as they operate on the same columns. Notice that:

$$T_5 \circ T_1 = T_2 \circ T_5$$
$$T_5 \circ T_2 = T_1 \circ T_5$$

  From this it is always possible to re-write Q as a composition of transforms where all T_5 transforms are always applied either first or last; and the sequence of applications T_5 can be reduce to either one or no applications of T_5 as this is an involution. The remaining of the sequence of transforms can be treated as P, which can be written as a sequence of unique transforms by strictly increasing indices. Therefore Q will be found by the generation scheme.

□

As there are five equivalences, $|[M]| \leq 2^5 = 32$. By generating all 32 permutation using Proposition 2 we ensure all equivalent states are found, though some may duplicate. We can generate these in order in such a way that the first 16 elements do not use T_5 while the second 16 do use T_5, implying that knowing if our permutation is in the first of second half of our list, we know which inverse scheme to use: P if it is in the first half, and $T_5 \circ P \circ T_5$ if it is in the second half of the list.

TODO

Enumerate combinatorially how many valid representative MDP states exist. Due to the five equivalences, I expect this to be around $2^5 = 32$ times smaller, not including symmetric states.

5.2 Actions

Define an action as an operator on an MDP state $s \in S$ to a ward state $M \in \mathcal{M}$. An action is placing a newly arriving patient into a bed, either unoccupied or occupied. If placing into an occupied bed, then the patient currently occupying that bed needs to be moved either into an unoccupied bed or into surge. We can denote an action by a pair of integers (b, d). When $b = d$ this denotes placing the newly arriving patient into bed b where that bed is unoccupied. When $b \neq d$ this denotes placing the newly arriving patient into bed b, and moving patient occupying bed b to bed d. When $d = 16$ this denotes moving the patient to surge.

For example, action $a = (3, 6)$ represents placing a patient into bed 3, and moving the patient currently in bed 3 to bed 6, e.g.:

$$a(M^\star, 2) = \left(\begin{array}{cccc:cccc:cccc:cccc} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 2 \end{array}\right) \tag{4}$$

where $M^\star$ is the ward state defined in Equation 1.

Actions are state-dependent:

- only Stage 2 patients can be moved into surge;
- arriving Stage 3-I patients *must* be placed into an isolation unit if free, or they *must* move Stage 2 or Stage 3 patients out of an isolation unit if no isolation unit is free. When there is a choice of patient type to move out of an isolation unit, Stage 2 patients are moved.

The set of actions available when in a particular state $s = (M, p)$, denoted by A_s, will depend on the ward state component, M and the arriving patient type p, such that the constraints above hold. That is:

$$A_s = \{(b, d) \mid b \in \{0, \ldots, 15\},\, d \in \{0, \ldots, 16\}\} \tag{5}$$

Subject to:

$$\begin{cases} b = d \implies M_{\cdot,b} = 0 & (6) \\ b \neq d \text{ and } d < 16 \implies M_{\cdot,d} < K_d \text{ and } M_{\cdot,b} > 0 \text{ and } M_{p,b} = 0 & (7) \\ d = 16 \implies p \neq 0 \text{ and } M_{0,b} > 0 & (8) \\ p = 2 \text{ and } M_{\cdot,15} < 2 \implies b = d = 15 & (9) \\ p = 2 \text{ and } M_{\cdot,15} = 2 \text{ and } M_{2,15} \neq 2 \implies b = 15 \text{ and } (M_{\cdot,d} = 0 \text{ or } d = 16) & (10) \\ p = 2 \text{ and } M_{\cdot,15} = 2 \text{ and } M_{2,15} = 2 \implies b \neq 15 \text{ and } (M_{\cdot,d} = 0 \text{ or } d = 16) & (11) \end{cases}$$

where $K_j = 1$ for all $j < 15$, and $K_{15} = 2$. Constraint 6 corresponds to placing a newly arriving patient in an empty bed without any bed moves; constraint 7 corresponds to placing the newly arriving patient into bed b, moving the current occupier to bed d; constraint 8 corresponds to only being able to move a Stage 2 patient to surge; constraint 9 corresponds to needing to place a newly arriving Stage 3-I patient into a free isolation unit; and constraint 10 corresponds to needing to move a less severe patient from an isolation unit to make room for an arriving Stage 3-I patient, and constraint 11 allows placing a Stage 3-I patient in another bed only if the isolation units are full with other Stage 3-I patients. Note that constraints 7 and 8 also restrict displacing a patients of the same type, as this would result in an equivalent state to placing them directly in an empty bed.

TODO

Enumerate combinatorially how many valid MDP states-action pairs exist.

These action sets are defined for all states $s \in S$, however due to state equivalences, we only need to define them for all $s \in S^\star$. When in state $s \neq S^\star$, we transform s my permutation P into a representative of its equivalence class, such that $P(s) \in S^\star$, Now a policy will then give an action a, which only applies to $P(s)$ and not s; to transform the action back, we use the inverse $P^{-1}(a)$ to get the corresponding action for s; where the inverse is given in Section 5.1.1.

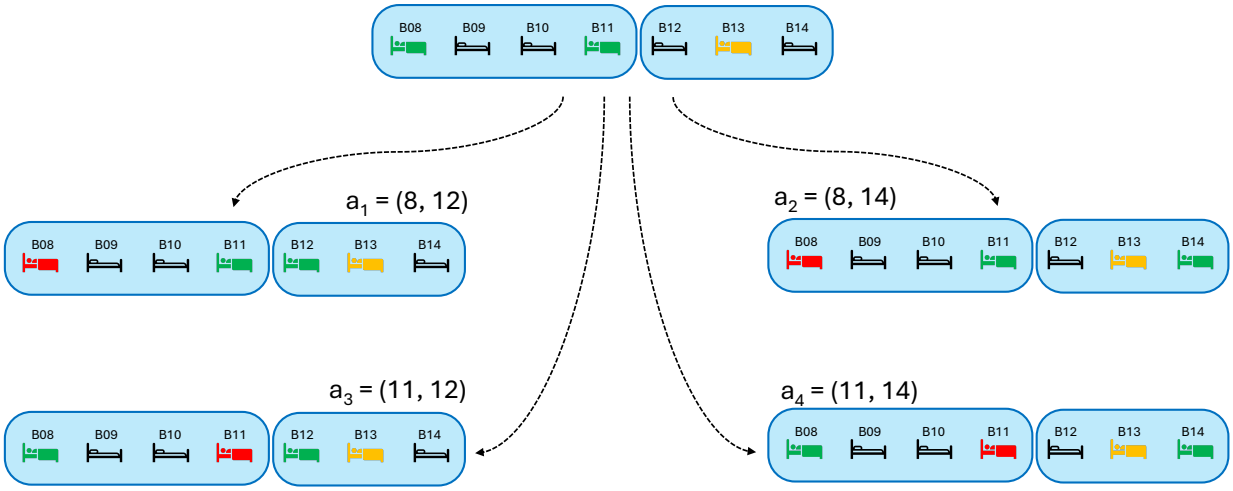


Figure 6: Example of an action equivalence.

5.2.1 Action Equivalences

If two actions transform a state two different yet equivalent matrices, we only need to consider one of these actions. For example consider actions $a_1 = (8, 12)$, $a_2 = (8, 14)$, $a_3 = (11, 12)$, and $a_4 = (11, 14)$ operating on a state $s = (M, 2)$, where Figure 6 shows beds B08 to B14 of M, before and after applying these actions - the resulting states are equivalent due to transforms T_3, T_4 and $T_3 \circ T_4$.

Therefore define a relation $\equiv$ such that $a_1 \equiv a_2$ if $a_2(s) \sim a_1(s)$, that is if the ward states they produce belong to the same equivalence class. It is clear that $\equiv$ also satisfies reflexivity, symmetry, and transitivity, and so is also an equivalence relation. Hence each set A_s can be partitioned into disjoint equivalence classes $[a_1] = \{a_2 \in A_s \mid a_2 \equiv a_1\}$, and so we can define $A_s^\star \subseteq A_s$ to be the set containing exactly one representative action from each equivalence class. How this representative action is chosen is detailed in Section 6.1.1.

Proposition 3. *For a state $s = (M, p)$, A_s can contain two equivalent actions when acting on s if and only if M is a fixed point of some permutation P, that is $P(M) = M$. Under this condition, any action $a = (b, d)$ where either b or d is affected by P has an equivalent action $a' = (P^{-1}(b), P^{-1}(d))$.*

As a preliminary to the proof, consider that applying an action to a state results in a matrix that is the sum of the original matrix plus some sparse difference matrix with elements in $\{-1, 0, 1\}$; consider the example matrix $M^\star$ with $a = (3, 6)$, we have $a(M^\star, 2) = M^\star + \delta_a$, where the matrix δ_a corresponds to removing a Stage 2 patient from bed B03, inserting a Stage 2 patient into bed B06, and inserting a Stage 3-I patient into bed B03:

$$\delta_a = \left(\begin{array}{cccc:cccc:cccc:cccc} 0 & 0 & 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}\right) \tag{12}$$

Note that if M is a fixed point of multiple permutations, $P_1, P_2, \ldots P_n$, then $a, P_1(a), P_2(a), \ldots P_n(a)$ are all equivalent, though not necessarily unique.

Proof. Two actions, a and a' are equivalent if, for some permutation P:

$$a(M) = P(a'(M)) \tag{13}$$
$$M + \delta_a = P(M + \delta_{a'}) \tag{14}$$
$$M + \delta_a = P(M) + P(\delta_{a'}) \tag{15}$$
$$M - P(M) = P(\delta_{a'}) - \delta_a \tag{16}$$

where Equation 15 is possible due to column permutations being linear transforms. For this to be true:

- either $P(M) = M$, and so M is a fixed point of P, and $P(\delta_{a'}) = \delta_a$, implying that $P(a') = a$, and so $a' = P^{-1}(a)$;
- or $P(M) \neq M$. In this case the left hand side of Equation 16 must have the property that permuted columns have opposite signs. For the same property to occur in the right hand side, we must have $a' = P(a)$, but this would imply that both sides of the equation are equal to zero, hence $P(M) = M$, which is a contradiction.

Hence equivalent actions can only arise under permutations P for which M is a fixed point, and $a = (b, d) \equiv a' = (P^{-1}(b), P^{-1}(d))$. In order for $a \neq a'$, then either b or d must be columns affected by P. □

TODO

Enumerate combinatorially how many valid representative MDP states-action pairs exist.

5.3 Rewards

The reward received for transitioning from state s_1 to state s_2 given action a was taken, $R(s_1, s_2, a)$, consists of four components:

$$\mathrm{R}(s_1, s_2, a) = -(C(s_1) + P(s_1, \theta))t_{s_1,s_2} - \Psi(a) \tag{17}$$

Where

- t_{s_1,s_2} is the time spent in state s_1 before transitioning to state s_2;
- $C(s_1)$ is the instantaneous staffing cost, corresponding to the number of staff required to care for the patients per time unit. This can be calculated from the ward-state component of s_1, given by Equation 18.
- $P(s_1, \theta)$ is the instantaneous penalty, measured in staff per time unit, corresponding to a Stage 3-I patient not being in an isolation unit. This can be calculated from the ward-state component of s_1, given by Equation 19, where θ is some penalty parameter.
- $\Psi(a)$ is the cost of taking action a, corresponding to either an intra-ward move for patients of type p (cost σ_p), an inter-ward move for patients of type p (cost ϕ_p), or moving a Stage 2 patient to surge (cost ξ). This is given by Equation 20.

$$C = G(M) + \sum_{i=1}^{2} \sum_{j=0}^{16} M_{i,j} \tag{18}$$

$$P(s_1) = \theta \sum_{j=0}^{15} M_{2,j} \tag{19}$$

$$\Psi(a) = \begin{cases} 0 \text{ if } b = d \\ \sigma_p \text{ if } b \text{ in the same ward as } d \\ \phi_p \text{ if } b \text{ in a different ward to } d \\ \xi \text{ if } d = 16 \end{cases} \tag{20}$$

where $s_1 = (M, p)$, $a = (b, d)$, and $G(M)$ represents a function that finds the minimum number of nurses required to cover the Stage 2 patients, when pairs of adjacent patients can be seen by the same nurse, as shown in Figure 4.

5.4 Simulation Model

The simulation model is parametrised as described in Section 3. Additionally we parametrise the isolation penalty with $\theta = 8$.

6 Reinforcement Learning Framework

Reinforcement learning is a model-free unsupervised learning that allows a central agent to learn optimal policies as it explores the state space during a simulation run. One standard algorithm is Q-learning, with overviews given in [9] and [10]. The idea is that each MDP state and action pair, (s, a) with $s \in S$ and $a \in A_s$, has some value $Q(s, a)$, called Q-values, representing the expected long term reward for taking action a when in state s, and following an optimal policy from then on. An optimal policy is, for each state, the action that gives the maximum expected reward, given by Equation 21.

$$a^\star = \underset{a \in A_s}{\operatorname{argmax}}\, Q(s, a) \tag{21}$$

The Q-values are found by iteratively updating them, as the simulation visits states, performs actions, and observes some reward, until convergence. The update, implemented once state s' has been reached, after taking action a in state s, and observing a reward $R(s, s', a)$ is given in Equation 22.

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha \left(R(s, s', a) + \gamma \max_{a' \in A'_s} Q(s', a') \right) \tag{22}$$

Here $\alpha \in (0, 1]$ is the learning rate, a parameter that indicates how important new information is, or how quick the agent is to trust new information; and $\gamma \in [0, 1)$ is the discount factor, a parameter that indicates how important future rewards are as opposed to immediate rewards. The ICU ward is stochastic, and so we anticipate a medium to low learning rate α would be required to overcome the variability in the rewards. The aim is for a general state of high reward throughout the ICU's operating times, and so future rewards are just as important as immediate rewards, so we anticipate a high discount factor γ would be beneficial. In particular, as the rewards are dependent on the time between updates (Equation 17), immediate rewards

are far less meaningful than long run average rewards - for example, one long interval with low instantaneous reward may look better than many shorter periods of high rewards, but it is the long run average that matters.

The reinforcement learning framework used is developed in Python. Due to the very high number of possible states-action pairs, the framework utilises high performance libraries such as Numpy [2] and Numba [7], and was optimised for both speed and memory with help from conversations with Google Gemini [4] to learn and steer some of the software development. It is fully unit-tested and verified, and openly available at `https://github.com/geraintpalmer/bed_moves_policy`.

The Q-values are stored in a Numba typed dictionary, an implementation of a hash table, that maps state-action pairs to Q-values, which are generally kept in memory. At the beginning of a run, all state-action pairs are unexplored, and the Q-value dictionary is empty. When state-action pairs are visited for the first time, the state-action pair is added to the dictionary, mapping it to the Q-value calculated using Equation 22, however this may require the Q-value of other unexplored state-action pairs. In these cases, the overall average Q-value found up to that point is used as a default value, $\bar{Q}$, which can be obtained by keeping track of the overall average reward, $\bar{R}$, using Equation 23.

$$\bar{Q} = \frac{\bar{R}}{1-\gamma} \tag{23}$$

6.1 State Hashing

Due to the vast number of potential state-action pairs, storing full tuples and matrices in the Q-table is very memory intensive. Therefore in order to preserve memory, we define a reversible hash between state-action pairs $(s, a) = ((M, p), a)$ and 64-bit integers (from -2^{63} to $2^{63} - 1$), that is a 19-digit integer.

We do this with the follow form of mixed radix encoding, given by Equation 24, where $\langle A, B\rangle_F = Tr(A^T B)$ is the Frobenius inner product [1], and the matrix W is given in Equation 25.

$$H((M, p), (b, d))) = 100000\langle M, W\rangle_F + 10000p + 100b + d \tag{24}$$

$$W = \begin{pmatrix} 3\times19\times4^{14} & 2\times19\times4^{14} & 1\times19\times4^{14} \\ 3\times19\times4^{13} & 2\times19\times4^{13} & 1\times19\times4^{13} \\ 3\times19\times4^{12} & 2\times19\times4^{12} & 1\times19\times4^{12} \\ 3\times19\times4^{11} & 2\times19\times4^{11} & 1\times19\times4^{11} \\ 3\times19\times4^{10} & 2\times19\times4^{10} & 1\times19\times4^{10} \\ 3\times19\times4^{9} & 2\times19\times4^{9} & 1\times19\times4^{9} \\ 3\times19\times4^{8} & 2\times19\times4^{8} & 1\times19\times4^{8} \\ 3\times19\times4^{7} & 2\times19\times4^{7} & 1\times19\times4^{7} \\ 3\times19\times4^{6} & 2\times19\times4^{6} & 1\times19\times4^{6} \\ 3\times19\times4^{5} & 2\times19\times4^{5} & 1\times19\times4^{5} \\ 3\times19\times4^{4} & 2\times19\times4^{4} & 1\times19\times4^{4} \\ 3\times19\times4^{3} & 2\times19\times4^{3} & 1\times19\times4^{3} \\ 3\times19\times4^{2} & 2\times19\times4^{2} & 1\times19\times4^{2} \\ 3\times19\times4^{1} & 2\times19\times4^{1} & 1\times19\times4^{1} \\ 3\times19 & 2\times19 & 1\times19 \\ 9 & 3 & 1 \end{pmatrix}^T \tag{25}$$

The first 15 columns use a counting scheme in base 4, as there are only 4 configurations for each of these columns. Now consider the 15th column, the entries are integers in $\{0, 1, 2\}$, and so the elements of this column can be counted in base 3. Noting however, due to the constraint that the sum of this column cannot

exceed 2, that the maximum value of this column would be 18, corresponding to $\binom{2}{0}{}_0$, then we can use base 18 to count this entire column. Together then, M can be represented by by numbers between 0 and 20401094655, that is by at most an 11 digit number. Now using positional concatenation (base 10) we can including the arriving patient type p and action $a = (b, d)$ gives us a 16 digit number, well within the confines of 64-bit integers.

As an example $H((M^\star, 2, (11, 4)) = 2025762603821104$, where $M^\star$ is the ward state given in Equation 1.

6.1.1 Choosing Representative States and Actions

In Sections 5.1.1 and 5.2.1 we introduced the idea of representative ward states and representative action, that can stand in place for all ward states and all actions in the same equivalence class. During the simulation run, whenever the algorithm encounters a state, the algorithm will update the Q-value for its equivalent representative state, ensuring faster learning and a smaller exploration space.

The representative states for a particular equivalence class are chosen as those states with the smallest hash. Similarly for actions.

Practically, this means that whenever the simulation reaches state M, then the hash for all states in $[M]$ are found, and the minimum is chosen. Similarly for actions.

6.2 Training Framework

During the simulation's training run, actions are selected using the ϵ-greedy policy [9], that is, for some probability $\epsilon \in [0, 1]$, the agent will choose the best discovered action so far, using Equation 21, with probability ϵ, and choose a random action with probability $1-\epsilon$. That is, the agent *explores* with probability $1 - \epsilon$, and *exploits* with probability ϵ.

Exploration versus exploitation is an important concept in reinforcement learning, especially in such vast state-spaces. Exploration, or a low ϵ, is important to be able to discover new state-action pairs, however spending too much time exploring can be detrimental, as it wastes time and resources on discovering states that would be seldom visited when following the optimal policy. Exploitation, or a high ϵ, ensures that relevant states are revisited many times, helping to gain enough information on these state-action pairs that their Q-values are updated enough times for these to be a good approximation of the expected future rewards. We choose to begin the training run with a low ϵ and gradually increase is throughout the run - exploring early to find plausible optimal policies, and exploiting later to refine them.

With such a vast state space, and high stochasticity, convergence for all states is practically impossible, and we must settle on very long enough run times to obtain good-enough Q-values to read off optimal policies. These long run times are still computationally expensive and time intensive. We therefore train the agent in a parallel framework, visualised in Figure 7.

We train the model over M stages, utilising N parallel workers. At stage $m = 0$, Q-value tables are initialised as empty, that is no states-action pairs have been discovered yet, and we set $\epsilon = 0$ for full exploration. These Q-value tables are replicated N times, and split over N parallel workers, each running its own training simulation, updating its own table of Q-values for a particular stretch of time. Once these parallel training runs are complete, each worker will have its own distinct table of Q-values: let $Q_n(s, a)$ represent the n^{th} worker's estimate of the Q-value for the (s, a) state-action pair. Workers also keep track of the number of visits to each state-action pair, let $h_n(s, a)$ be the number of times worker n visited that state-action pair. These tables are then combined, weighting each Q-value by the number of visits, that is:

$$Q(s, a) = \frac{\sum_{n=0}^{N} Q_n(s, a) h_n(s, a)}{\sum_{n=0}^{N} h_n(s, a)} \tag{26}$$

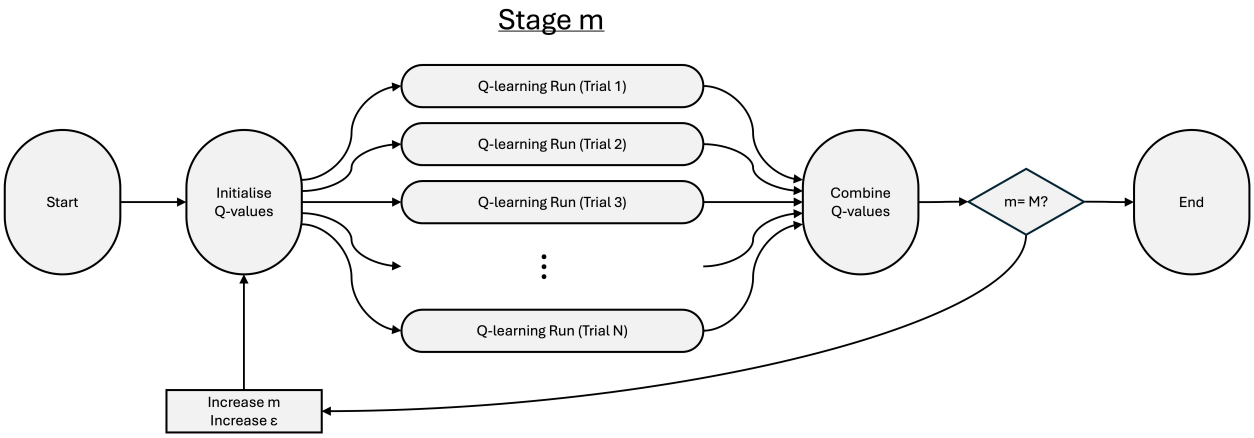


Figure 7: Framework used for training the model.

In the next state, we update $\epsilon \leftarrow \epsilon + 1/M$, gradually increasing the agents rate of exploitation. Then Q-values are initialised with the combined Q-value tables found at the end of the last stage, the h_n are reset to zero, and the training is repeated on parallel workers. This ensures that at the first stage we have full exploration, and by the final stage we have full exploitation, with ϵ increasing linearly over the stages.

After each stage the combined Q-values are saved and so the incumbent policy can be evaluated by running the simulation, without any Q-learning, choosing actions based on that incumbent policy.

6.3 Pruning

As the number of theoretically possible state-action pairs is so large, the memory constraints of a computer makes running the algorithm difficult in practice, as the Q-value table grows with exploration. However, the practically reachable space of state-action pairs, when following an optimal, or even just a reasonable, policy, is much smaller. This is often called the 'on-policy distribution' [9]. Therefore any state-action pairs in the suboptimal trajectory space, that is those not in the on-policy distribution, can generally be ignored, reducing the Q-value table and saving memory.

After the end of each stage, these states are identified as those who have not been visited frequently, if at all, and so can be pruned. We prune all state-action pairs whose number of hits $h_n(s, a)$ is less than some chosen limit $\tilde{h}$. This not only helps save memory, but also cleans the data: the simulation is highly stochastic, and for states with $h_n(s, a) < \tilde{h}$, its Q-value will have been based on $h_n(s, a)$ observations only, meaning that for very small $\tilde{h}$ this is just stochastic noise and so cannot meaningfully estimate the expected rewards.

Note that due to the way in which Q-values are passed from stage to stage, it is possible to reach the end of a stage and for a particular state-action pair to be present in the table but have been visited zero times: for example if a pair was visited in stage $n-1$, it was passed to stage n and its number of hits reset to zero, but then not visited at all in stage n. This would be a good candidate to prune and so not pass to stage $n+1$.

7 Experiments

For a particular run, we evaluate each stage' incumbent policy by running the simulation for 60 trials, for a 30,000 time units, and a warm-up time of 5,000 time units, and recording the overall cost running the ICU for the remaining 25,000 time units when exploiting that incumbent policy. An initial 'random' policy is evaluated for comparison.

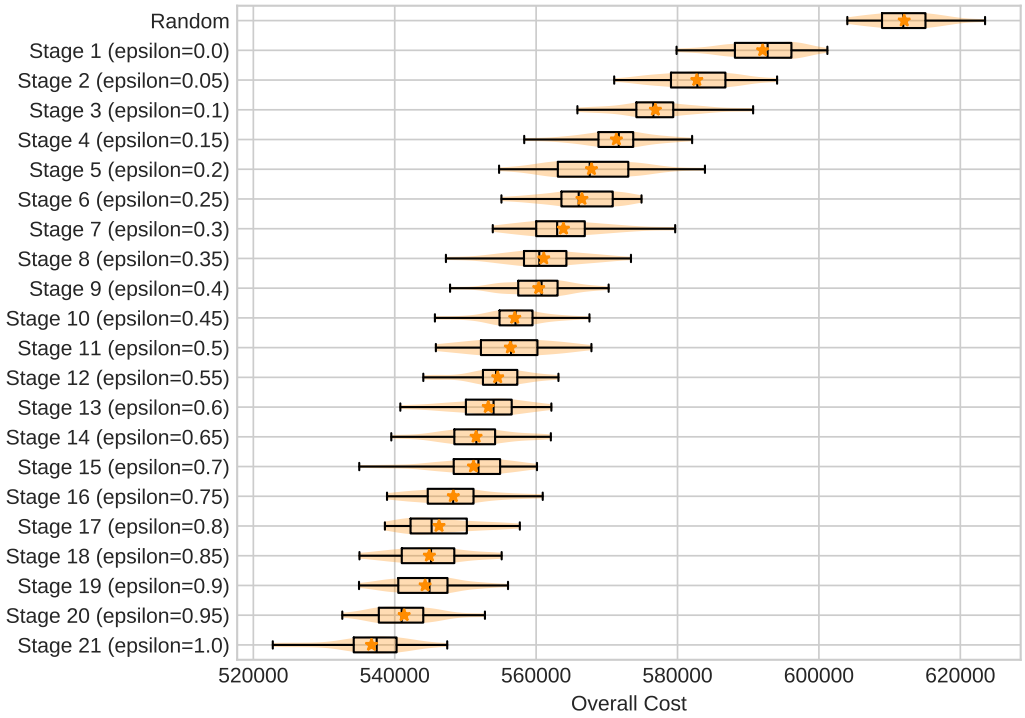


Figure 8: Evaluation of the incumbent policies across the stages.

First we train the agent using the framework described in Section 6, choosing $\alpha = 0.6$ and $\gamma = 0.6$. During training each stage was run for 500,000 time units, using 20 parallel workers per stage. Figure 8 shows these results. We see that as the stages increase (and hence more training), the agent makes very quick gains in the early stages as the agent explores more, and then slower gains as the agent exploits and refines the policies it has already learned. In this case the agent has made gains from a cost of 366681.25 using the random policy, to 273569.88 using the policy at the end of the 21 stages, an improvement of 25.4%.

As there is such a huge amount of state-action pairs, it is practically impossible to estimate whether the policy has converged or not, though we see any improvements in cost slowing down as the stages increase. We can also count the number of unique state-action pairs visited per stage during the training process, shown in Figure 9. We see some tapering-off of the number of new states-action pairs visited, indicating both a decreasing tendency to explore (from the increasing ϵ), and a gradual saturation of states from previous explorations.

7.1 Effect of the learning rate α

Using this setup then, we can also look at the effect of the learning rate α. Figure 10 shows the percentage improvement in using the incumbent policy per stage for different values of α (left), and the percentage improvement after all 21 stages for different values of α (right).

We immediately see from the left plot that the improvement peaks when $\alpha = 0.6$, indicating that this is the ideal learning rate for this problem, when running under the current setup. However the right plot shows the agent's behaviour as the training progresses: a smaller learning rate results in slower learning (a less steep curve); a high learning rate has faster learning, but cannot smooth out stochasticity of rewards as effectively; while a moderate learning rate can learn quickly and smooth out variability. The speed of learning

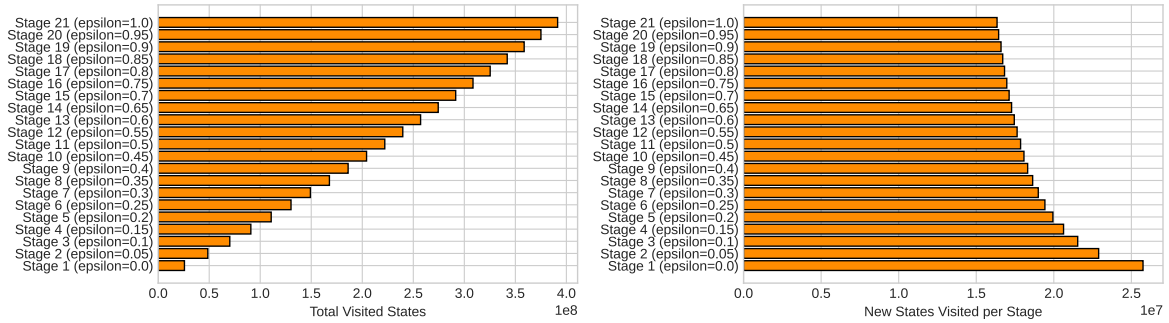


Figure 9: Number of state-action pairs visited in each stage of the training process.

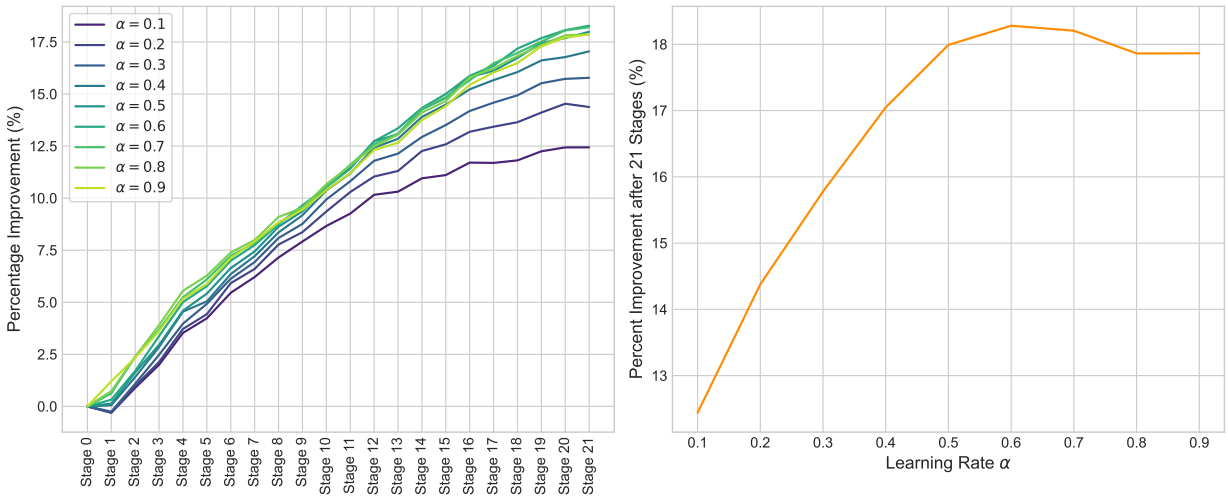


Figure 10: The effect of varying the learning rate α.

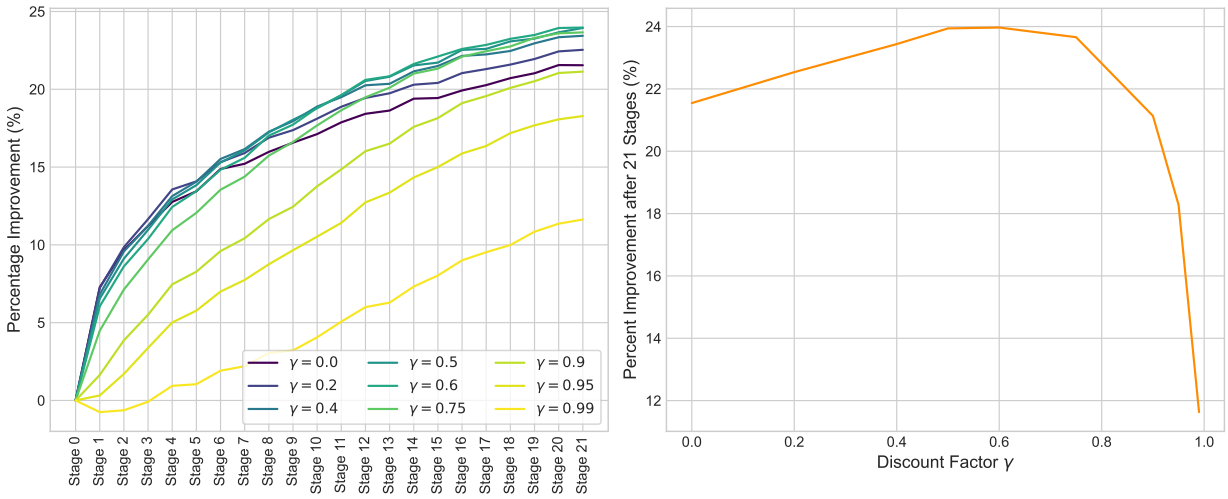


Figure 11: The effect of varying the discount factor γ.

is important in our case is important: as it would be impractical to run the algorithm until convergence we have to choose some temporal cut-off, and so we want as much learning to occur in the time allotted as possible.

7.2 Effect of the discount factor γ

We can also look at the effect of the discount factor γ. Figure 11 shows the percentage improvement in using the incumbent policy per stage for different values of γ (left), and the percentage improvement after all 21 stages for different values of γ (right).

Again we can immediately see from the plot on the left that the improvement peaks when γ is around 0.5-0.6. Considering the right plot which shows the agent's behaviour as the training progresses: a high discount factor results in slower learning (a less steep curve), this is due to each update requiring a larger input from future rewards, which themselves need to be well-learned; a small discount factor has faster learning, but account for steady-state or average long-run rewards as effectively. Again, the speed of learning due to the need to choose some arbitrary temporal cut-off.

7.3 Effect of the traffic intensity

Bed moves policies might have a greater impact on some ICU wards over other, or there might be some situation where finding optimal bed moves policies can have greater impact. One feature that is likely to have an effect is the ward's business, or the traffic intensity of the ward. To test this, introduce some factor m, and multiply all arrival rates by m, while keeping the service rates fixed. In this way increasing m increases the traffic intensity ρ of the ward linearly, due to summation of traffic intensities across customer classes:

$$\rho = \rho_{\text{green}} + \rho_{\text{amber}} + \rho_{\text{red}} = \frac{m}{17}\left(\frac{\lambda_{\text{green}}}{\mu_{\text{green}}} + \frac{\lambda_{\text{amber}}}{\mu_{\text{amber}}} + \frac{\lambda_{\text{red}}}{\mu_{\text{red}}}\right) \tag{27}$$

To keep costs and training epochs relatively comparable, we reduce the maximum training and evaluation times by a factor m also. Figure 12 shows the overall evaluation cost, ad ρ varies, for each stage of training.

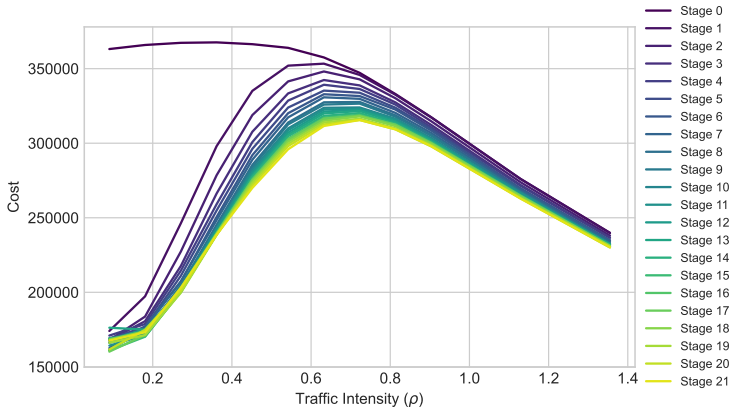


Figure 12: The effect of varying the traffic intensity, ρ.

When ρ is small, we see that there are huge gains to be made, and that most of these gains happen in the early stages, that is after not much training. This implies that these are easy or obvious changes that can be made using human instinct, and a reinforcement learning algorithm may not be needed in practise. The large gains may also show that in such an empty ward, it is easy to reach an 'ideal' scenario, where no penalties or extra staffing is required, as the near empty system is not very complex.

When ρ is very large, there is hardly any gains to be made. This may be because ward is so busy that any bed move actions are not helpful, and penalties and extra staffing are unavoidable whatever moves are taken. Here there is not enough 'wiggle room' to influence much.

This implies that the ideal times to invest in a bed moves policy is for moderate traffic intensities, where $0.3 < \rho < 0.6$. Here large to moderate gains are still possible, however these gains occur in the later stages of training, implying they are less obvious and hence the power of the reinforcement learning algorithm is useful. This is a sweet spot where there is sufficient complexity in the system to require a policy, and enough 'wiggle room' to make substantial gains.

8 Human-Readable Policies

TODO

Idea: Currently the policies that come from the reinforcement learning is a mapping from millions / tens of millions of states to optimal actions. I wonder if we can use some sort of machine learning / clustering algorithm to simplify this into general human-readable 'guidelines' rather than a full detailed policy. This simplified policy would of course be less effective than the full policy, but this is something that can be measured and analysed.

References

[1] Jeff Calder and Peter J Olver. *Linear Algebra, Data Science, and Machine Learning*. Springer, 2025.

[2] Harris CR et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[3] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2004.

[4] Google DeepMind. Gemini (3 flash), 2026.

[5] Varun Gupta, Mor Harchol Balter, Karl Sigman, and Ward Whitt. Analysis of join-the-shortest-queue routing for web server farms. *Performance Evaluation*, 64(9-12):1062–1081, 2007.

[6] John FC Kingman. Two similar queues in parallel. *The Annals of Mathematical Statistics*, 32(4):1314–1323, 1961.

[7] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015.

[8] Geraint I Palmer, Vincent A Knight, Paul R Harper, and Asyl L Hawa. Ciw: An open-source discrete event simulation library. *Journal of Simulation*, 13(1):68–82, 2019.

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.

[10] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.